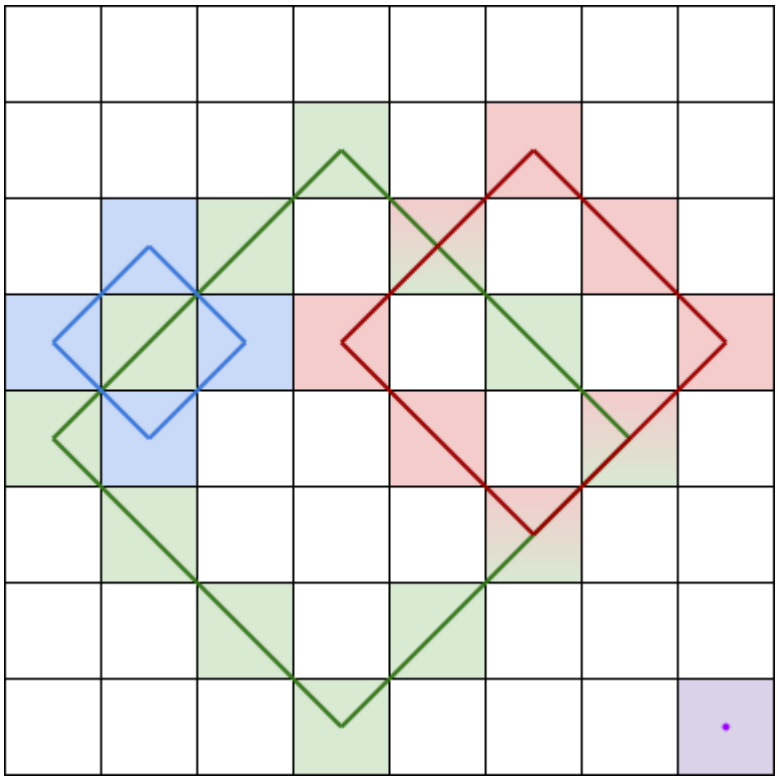
1878. Get Biggest Three Rhombus Sums in a Grid

You are given an m x n integer matrix grid.

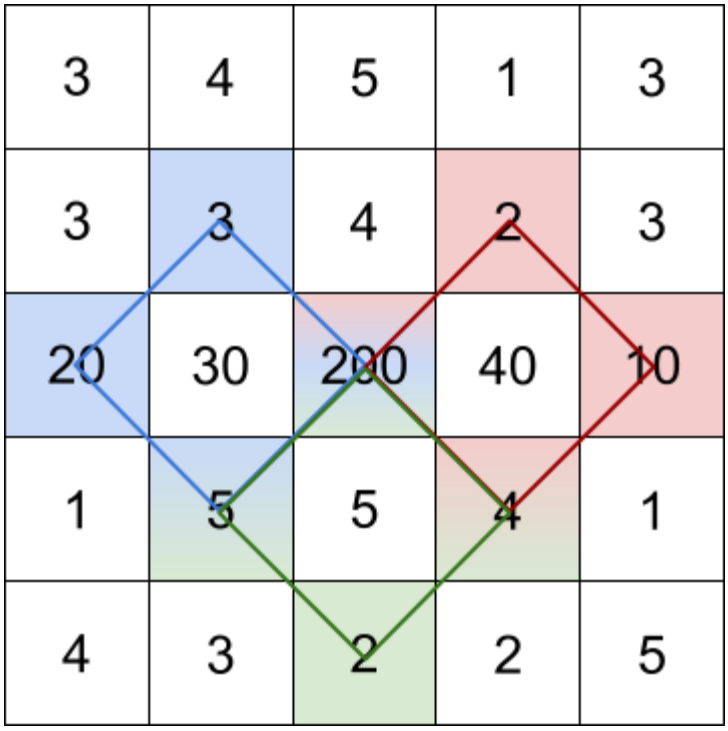
A **rhombus sum** is the sum of the elements that form **the border** of a regular rhombus shape in grid. The rhombus must have the shape of a square rotated 45 degrees with each of the corners centered in a grid cell. Below is an image of four valid rhombus shapes with the corresponding colored cells that should be included in each **rhombus sum**:



Note that the rhombus can have an area of 0, which is depicted by the purple rhombus in the bottom right corner.

Return *the biggest three* ***distinct rhombus sums*** *in the* grid *in* ***descending order****. If there are less than three distinct values, return all of them*.

Example 1:



Input: grid = [[3,4,5,1,3],[3,3,4,2,3],[20,30,200,40,10],[1,5,5,4,1],[4,3,2,2,5]]

Output: [228,216,211]

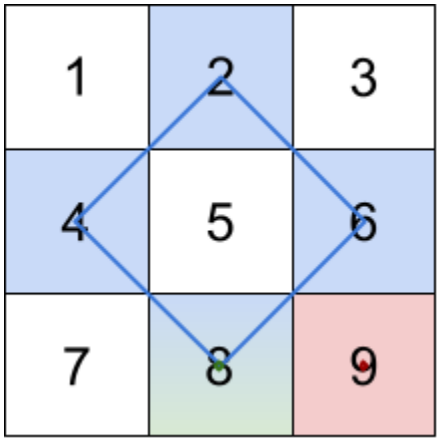
Explanation: The rhombus shapes for the three biggest distinct rhombus sums are depicted above.

- Blue: 20 + 3 + 200 + 5 = 228

- Red: 200 + 2 + 10 + 4 = 216

- Green: 5 + 200 + 4 + 2 = 211

Example 2:



Input: grid = [[1,2,3],[4,5,6],[7,8,9]]

Output: [20,9,8]

Explanation: The rhombus shapes for the three biggest distinct rhombus sums are depicted above.

- Blue: 4 + 2 + 6 + 8 = 20

- Red: 9 (area 0 rhombus in the bottom right corner)

- Green: 8 (area 0 rhombus in the bottom middle)

Example 3:

Input: grid = [[7,7,7]]

Output: [7]

Explanation: All three possible rhombus sums are the same, so return [7].

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 50
- 1 <= grid[i][j] <= 10^5

Solution:---

```
class Answer {

   int[] ans;

   public Answer() {
      ans = new int[3];
   }

   void put(int x) {
      if (x > ans[0]) {
```

```
            ans[2] = ans[1];
            ans[1] = ans[0];
            ans[0] = x;
        } else if (x != ans[0] && x > ans[1]) {
            ans[2] = ans[1];
            ans[1] = x;
        } else if (x != ans[0] && x != ans[1] && x > ans[2]) {
            ans[2] = x;
        }
    }

    List<Integer> get() {
        List<Integer> ret = new ArrayList<>();
        for (int num : ans) {
            if (num != 0) {
                ret.add(num);
            }
        }
        return ret;
    }
}

class Solution {

    public int[] getBiggestThree(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int[][] sum1 = new int[m + 1][n + 2];
```

```
int[][] sum2 = new int[m + 1][n + 2];

for (int i = 1; i <= m; ++i) {
    for (int j = 1; j <= n; ++j) {
        sum1[i][j] = sum1[i - 1][j - 1] + grid[i - 1][j - 1];
        sum2[i][j] = sum2[i - 1][j + 1] + grid[i - 1][j - 1];
    }
}

Answer ans = new Answer();
for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        // a single cell is also a rhombus
        ans.put(grid[i][j]);
        for (int k = i + 2; k < m; k += 2) {
            int ux = i;
            int uy = j;
            int dx = k;
            int dy = j;
            int lx = (i + k) / 2;
            int ly = j - (k - i) / 2;
            int rx = (i + k) / 2;
            int ry = j + (k - i) / 2;
            if (ly < 0 || ry >= n) {
                break;
            }
            int sum =
                (sum2[lx + 1][ly + 1] - sum2[ux][uy + 2]) +
```

```
                (sum1[rx + 1][ry + 1] - sum1[ux][uy]) +
                (sum1[dx + 1][dy + 1] - sum1[lx][ly]) +
                (sum2[dx + 1][dy + 1] - sum2[rx][ry + 2]) -
                (grid[ux][uy] +
                    grid[dx][dy] +
                    grid[lx][ly] +
                    grid[rx][ry]);
            ans.put(sum);
        }
    }
}

List<Integer> resultList = ans.get();
int[] result = new int[resultList.size()];
for (int i = 0; i < resultList.size(); i++) {
    result[i] = resultList.get(i);
}
return result;
}
}
```